LOVELY PROFESSIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC TITLE: MAJOR PROJECT REPORT

WANDERLUST

A Full-Stack Vacation Rental Platform with Document-Oriented NoSQL Architecture
Built with MongoDB Atlas, Express.js, Node.js, EJS, Passport.js, and Razorpay

NAME OF THE FACULTY -:	Dr. Manpreet Singh Sehgal (32354)
COURSE CODE -:	CSE494
COURSE TITLE -:	INTELLIGENT NOSQL DATABASES
SUBMITTED BY -:	**Vecha Laxmi Swaraj babu**
REGISTRATION NUMBER -:	**12307477**
ROLL NUMBER -:	**60**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNJAB, INDIA

ACADEMIC SESSION: 2025 – 2026

GITHUB LINK: https://github.com/Swarajbabu/Wanderlust-Travel-Stay-Booking-Application

ABSTRACT

Modern vacation rental platforms handle complex polymorphic property listings, high-concurrency reservation schedules, and secure monetary settlements. Conventional relational database management systems (RDBMS) often impose schema rigidity and costly multi-table joins when managing heterogeneous property attributes, nested multimedia assets, and geospatial coordinates at scale. This project presents **Wanderlust**, an end-to-end full-stack vacation rental web platform developed using the **HCJNE stack** (HTML5, CSS3, JavaScript ES6+, Node.js, Express.js) backed by a production-grade **MongoDB Atlas** NoSQL document database.

Wanderlust models data across four primary collections: **Users**, **Listings**, **Bookings**, and **Reviews**, alongside an independent distributed **Sessions** collection managed via **connect-mongo**. The database architecture emphasizes document modeling best practices, balancing normalized references (**ObjectId** associations with Mongoose population) and embedded subdocuments (GeoJSON Point definitions, Cloudinary image descriptors). Performance optimization is achieved via compound text indexing across multi-field listing metadata, atomic array mutations (such as **$pull** for review deletions), and Mongoose lifecycle hooks (including pre-save date range assertions and cascading **post("findOneAndDelete")** cleanup triggers).

The platform integrates four commercial third-party cloud services: **Mapbox GL** for forward geocoding and vector map rendering, **Cloudinary** for multipart image streaming and CDN delivery, **Razorpay** with HMAC-SHA256 signature verification for mathematical payment reconciliation, and **Nodemailer** for one-time password (OTP) verification and transactional booking notifications. Enterprise-grade web security is enforced through double-submit CSRF cookies, Helmet Content Security Policy headers, and Express rate limiters. System reliability is verified via automated integration tests executed against an isolated in-memory replica set (**mongodb-memory-server**) using Jest and Supertest.

ACKNOWLEDGMENT

I take this opportunity to express my profound gratitude and deep regards to my Project Supervisor and the faculty members of the Department of Computer Science & Engineering for their exemplary guidance, constant encouragement, and valuable suggestions throughout the development of this major project.

I am equally thankful to the open-source software community for providing the Node.js, Express, MongoDB, and Mongoose ecosystems, which formed the foundational pillars of this engineering endeavor.

TABLE OF CONTENTS

Certificate of Approval ii

Declaration iii

Abstract iv

Acknowledgment v

Chapter 1: Introduction 1

1.1 Background & Motivation 1

1.2 Problem Statement 1

1.3 Objectives of the Project 2

1.4 Scope and Limitations 2

1.5 Organization of the Report 3

Chapter 2: Literature Survey & Technology Stack 4

2.1 Existing Systems vs. Proposed System (RDBMS vs. NoSQL) 4

2.2 Core Tech Stack (HCJNE) 5

2.3 Supporting Services & Third-Party Integration Matrix 6

Chapter 3: Requirement Analysis & Specifications 7

3.1 Software & Hardware Requirements 7

3.2 Functional Requirements (FR) 8

3.3 Non-Functional Requirements (NFR) & Security Matrix 9

Chapter 4: System Architecture & Workflow Design 10

4.1 High-Level Architecture (MVC & Service Pipeline) 10

4.2 User Authentication & OTP Verification Workflow 12

4.3 Listing Creation & Mapbox Geocoding Workflow 13

4.4 Overlap-Guarded Booking & Razorpay Payment Verification Flow 14

4.5 Verified-Stay Review Submission Flow 16

Chapter 5: Database & Schema Design (MongoDB Deep Dive) 17

5.1 Why MongoDB for Vacation Rentals? (Embedding vs. Referencing) 17

5.2 Collection Schemas & Data Dictionaries 18

5.3 MongoDB Indexing Strategy & Execution Plan Analysis (explain()) 22

5.4 Mongoose Middleware, Lifecycle Hooks & Atomic Operators 24

Chapter 6: Module Design & Implementation 25

6.1 Authentication & Role-Based Access Control Module 25

6.2 Listing Management & Hybrid Search Engine Module 26

6.3 Booking Financial Engine & Razorpay HMAC Cryptography 27
6.4 Security, Sanitization & Validation Module 28
Chapter 7: Testing & Quality Assurance 29
7.1 Automated Testing Methodology 29
7.2 In-Memory Database Isolation (mongodb-memory-server) 29
7.3 Test Execution Matrix & Results 31
Chapter 8: Results, Deployment & Future Enhancements 32
8.1 Production Deployment Configuration 32
8.2 Future Enhancements 33
8.3 Conclusion 33
Appendix A: Sample MongoDB Document Exports (BSON) 34
Appendix B: Application Route Matrix 35
References 36

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

Over the last decade, the international tourism and hospitality industry has evolved rapidly from centralized hotel reservation chains toward decentralized peer-to-peer (P2P) residential rental networks, exemplified by platforms such as Airbnb and Vrbo. In this model, private property owners (hosts) publish residential assets directly to traveling consumers (guests). Standard hotel inventory consists of standardized, homogeneous units (e.g., standard deluxe, twin executive). Conversely, vacation rentals are intrinsically heterogeneous and polymorphic: they range from urban apartments and beachfront villas to rustic cabins, camping yurts, and houseboats.

Engineering a reliable web system to manage this domain requires solving several structural data persistence and security challenges. Property data includes semi-structured descriptors, multiple high-resolution photos, geographic coordinates, and dynamic categorization. Concurrently, reservation schedules demand strict temporal consistency to prevent double-booking anomalies, while financial checkout flows require server-authoritative tariff calculations to prevent client-side payment manipulation.

1.2 Problem Statement

Traditional Relational Database Management Systems (RDBMS) rely on rigid tabular schemas and formal referential constraints. When managing polymorphic vacation rentals, an RDBMS necessitates complex normalization—spreading property data across multiple tables (properties, coordinates, amenities, image galleries, category lookups) connected by foreign keys. On every read request, expensive SQL multi-table **JOIN** operations degrade response latencies.

Furthermore, booking systems are prone to race conditions and temporal overlap collisions. If two guests attempt to reserve overlapping date ranges concurrently, an unhedged database query may permit double-bookings. In addition, web platforms face security vulnerabilities including Cross-Site Request Forgery (CSRF), injection attacks, and credential stuffing. A production-ready vacation rental platform must address these operational, data modeling, and security challenges within a maintainable, high-performance architecture.

1.3 Objectives of the Project

The primary objective of **Wanderlust** is to design, develop, and benchmark a full-stack vacation rental platform centered on a document-oriented **MongoDB** NoSQL architecture. The specific technical objectives are:

- **Document-Oriented Persistence:** Design a hybrid-modeled MongoDB schema across four primary collections (**Users**, **Listings**, **Bookings**, **Reviews**) and an external **Sessions** store, optimizing read throughput while avoiding the 16 MB BSON document size boundary.
- **Atomic Date Overlap Prevention:** Formulate an atomic date-range filtering query in MongoDB to eliminate double-booking race conditions during reservation creation.
- **Authoritative Tariff & Cryptographic Payment Processing:** Implement server-side tariff computation (nights × nightly rate + 18% GST) and verify Razorpay payment integrity via HMAC-SHA256 signature validation.
- **Verified-Stay Review Ecosystem:** Restrict rating and review submission exclusively to authenticated guests who possess confirmed, completed past bookings for the specific listing.

- **Defense-in-Depth Application Security:** Enforce double-submit cookie CSRF defense, Helmet Content Security Policy directives, IP-based rate limiting, and Bcrypt-hashed OTP email verification.
- **Isolated In-Memory Automated Testing:** Validate core application workflows using Jest and Supertest against an ephemeral in-memory MongoDB replica set (**mongodb-memory-server**).

1.4 Scope and Limitations

The project encompasses user onboarding (local signup, Google OAuth 2.0, OTP email verification), property management with Cloudinary image uploading and Mapbox forward geocoding, real-time date availability checking, Razorpay checkout, booking lifecycle management (**pending → confirmed → cancelled**), and automated transactional emails.

Current boundaries exclude multi-currency foreign exchange conversion (transactions are standardized in INR), automated dispute arbitration, and multi-calendar synchronization via external iCal feeds.

1.5 Organization of the Report

This report is structured into eight chapters: Chapter 2 reviews literature and justifies the HCJNE stack and NoSQL database choice. Chapter 3 outlines hardware, software, functional, and non-functional specifications. Chapter 4 presents the high-level architecture and execution workflows. Chapter 5 examines MongoDB schema design, indexing, and Mongoose hooks. Chapter 6 details module implementations. Chapter 7 covers automated testing and in-memory database isolation. Chapter 8 presents production deployment, future directions, and conclusions.

CHAPTER 2: LITERATURE SURVEY & TECHNOLOGY STACK

2.1 Existing Systems vs. Proposed System (RDBMS vs. NoSQL)

Selecting an optimal database architecture is fundamental to vacation rental web platforms. Relational models (PostgreSQL, MySQL) were evaluated against document-oriented NoSQL databases (MongoDB).

Parameter	Relational Database (RDBMS)	MongoDB Document Store (Wanderlust)
Schema Flexibility	Rigid tabular schemas. Adding new listing attributes requires formal DDL schema migrations.	Dynamic BSON documents. Polymorphic listing attributes coexist naturally without schema locks.
Multimedia & Coordinates	Requires auxiliary junction tables for image galleries and specialized SQL GIS plugins.	Native embedded subdocuments for Cloudinary image arrays and native GeoJSON **Point** objects.
Query Overhead	Requires multi-table SQL **JOIN** operations across Listings, Users, Images, and Categories on read.	Read-optimized single document retrieval. Related documents populated via targeted **populate()** calls.
Full-Text Search	Requires external search daemons (Elasticsearch) or complex full-text indices.	Native multi-field compound text indexing (**$text**) with linguistic tokenization and stemming.
Session Persistence	Requires relational session tables or external Redis caches.	Distributed session persistence via **connect-mongo** leveraging native document TTL expiry.

MongoDB provides structural affinity for rental platforms where reads dominate writes, property media and coordinates are nested within parent entities, and schema polymorphism is common.

2.2 Core Tech Stack (HCJNE)

Wanderlust is implemented using the **HCJNE** architectural stack, representing a unified JavaScript runtime pipeline:

- **HTML5 & EJS (Embedded JavaScript Templates):** Presentation layer utilizing **ejs-mate** layouts for DRY (Don't Repeat Yourself) master boilerplates, navigation bars, and footers while dynamically rendering server-injected data.
- **CSS3 & Bootstrap 5:** Mobile-first responsive layout framework supplemented by custom stylesheets for date pickers, card hovering, and rating inputs.
- **JavaScript (ES6+):** Powers client-side map rendering (Mapbox GL), calendar logic (Flatpickr), and Razorpay checkout, alongside server-side asynchronous async/await workflows.
- **Node.js (v18.x - v22.x LTS):** Event-driven, non-blocking I/O runtime capable of handling high-concurrency HTTP requests.
- **Express.js (v4.21):** Minimalist web routing framework providing middleware chaining, route modularization, session management, and centralized error handling.

2.3 Supporting Services & Third-Party Integration Matrix

Service	Library / SDK	Responsibility in Wanderlust
MongoDB Atlas	**mongoose** (v9.7)	Cloud-hosted document persistence, automated failover replica sets, indexing, and transaction management.
Cloudinary	**cloudinary**, **multer-storage-cloudinary**	Direct streaming of multipart image uploads, format optimization (WebP), and global CDN image delivery.
Mapbox	**@mapbox/mapbox-sdk**, **mapbox-gl**	Forward geocoding textual location strings into GeoJSON Point coordinates and interactive vector map rendering.
Razorpay	**razorpay**, **crypto**	Server-side payment order generation, checkout modal invocation, and cryptographic HMAC-SHA256 signature verification.
Nodemailer	**nodemailer**	Automated delivery of 6-digit verification OTPs, password reset links, and booking confirmations via SMTP.

CHAPTER 3: REQUIREMENT ANALYSIS & SPECIFICATIONS

3.1 Software & Hardware Requirements

- **Hardware Requirements:**
  - Processor: Dual-Core 2.0 GHz CPU minimum (Intel Core i3 / AMD Ryzen 3 or higher).
  - RAM: 4.0 GB minimum (8.0 GB recommended for running in-memory test databases concurrently).
  - Storage: 500 MB free disk space for project source code, dependencies, and temporary assets.
  - Network: Active internet connection for Atlas, Mapbox, Cloudinary, and Razorpay APIs.
- **Software Requirements:**
  - Operating System: Cross-platform (Windows 10/11, Ubuntu Linux 20.04+, macOS 12+).
  - Runtime: Node.js (Version 18.0.0 through 22.x LTS).
  - Database: MongoDB Community Edition 6.0+ or MongoDB Atlas Cloud Cluster.
  - Web Browser: Any modern Chromium, WebKit, or Gecko browser supporting ECMAScript 2020+.

3.2 Functional Requirements (FR)

- **FR-1: User Lifecycle & Authentication Management:**
  - Users can register with username, email, and password. Passwords are salted and hashed via PBKDF2.
  - Mandatory email verification via a 6-digit random OTP valid for 10 minutes prior to listing creation.
  - Google OAuth 2.0 single sign-on with automatic account linking when emails match existing local accounts.
  - Time-limited (1-hour) cryptographic token workflow for password reset requests.
- **FR-2: Property Listing Management:**
  - Hosts can create, read, update, and delete property listings with titles, descriptions, prices, categories, and locations.
  - Automatic image streaming to Cloudinary and forward-geocoding of addresses into GeoJSON coordinates via Mapbox.
  - Only the designated listing owner can modify or delete their listing.
  - Search engine supporting text keywords, category filtering, and server-side pagination.
- **FR-3: Reservation & Payment Processing:**
  - API endpoint (**/booked-dates**) returning active reservation date intervals to disable calendar days in the UI.
  - Server-side validation rejecting inverted dates (**checkOut ≤ checkIn**) and overlapping bookings.
  - Server-calculated total pricing: **(nights × price) + 18% GST**, preventing client-side price tampering.
  - Razorpay order creation and HMAC-SHA256 signature verification prior to booking confirmation.
- **FR-4: Verified Stay Reviews:**
  - Review submissions are permitted only if the guest possesses a confirmed past booking on that listing with a checkout date in the past.

- Review authors can delete their own reviews; deleting a listing cascades deletion to all associated reviews.

3.3 Non-Functional Requirements (NFR) & Security Matrix

Security Dimension	Implementation Mechanism	Threat Mitigated
CSRF Protection	**csrf-csrf** double-submit signed cookie pattern	Prevents unauthorized cross-site form submissions and state-changing attacks.
Content Security Policy	**helmet** CSP headers with strict origin whitelisting	Mitigates Cross-Site Scripting (XSS) and unauthorized script injection.
Rate Limiting	**express-rate-limit** IP throttling on auth endpoints	Blunts brute-force login attempts, credential stuffing, and OTP flooding.
Payment Verification	Cryptographic HMAC-SHA256 signature calculation	Prevents client-side payment spoofing and unauthorized booking confirmation.
Password & OTP Hashing	PBKDF2 (Passport) and Bcrypt (10 salt rounds)	Ensures stored credentials and OTP codes cannot be reversed if database is read.

CHAPTER 4: SYSTEM ARCHITECTURE & WORKFLOW DESIGN

4.1 High-Level Architecture (MVC & Service Pipeline)

Wanderlust follows a layered Model-View-Controller (MVC) pattern with distinct security, routing, and controller tiers. The separation of concerns ensures that business logic, persistence, and presentation remain modular.

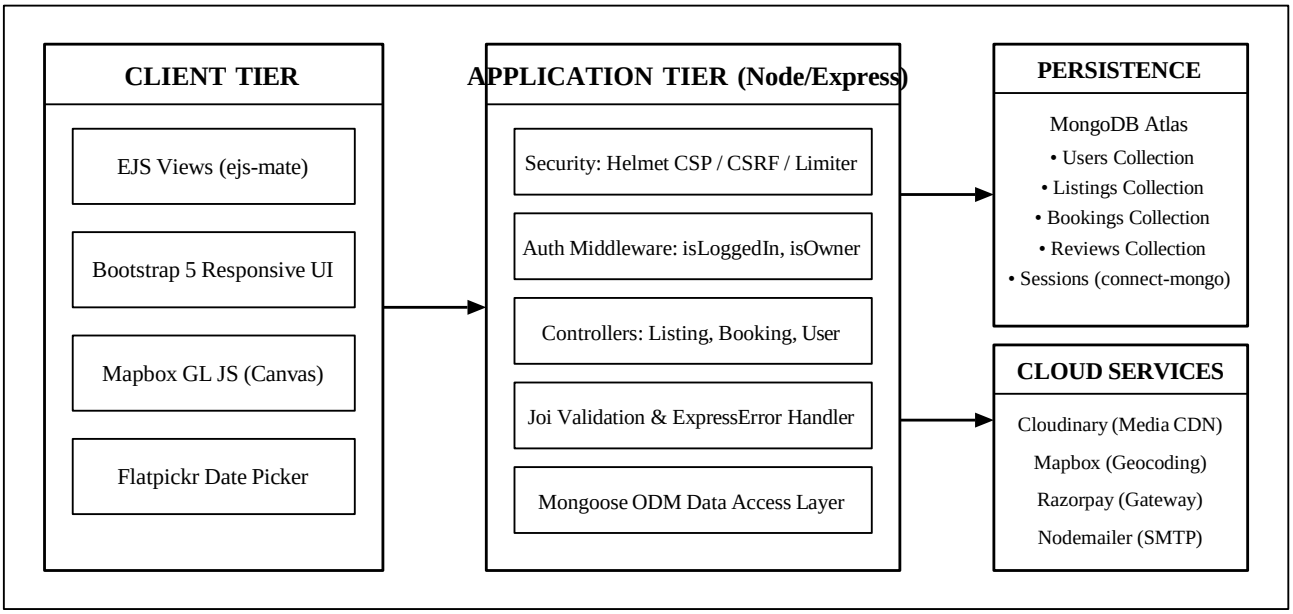


Figure 4.1: Overall System Architecture of Wanderlust showing client, middleware, application, and persistence tiers.

As depicted in Figure 4.1, requests originate at the client tier, traverse defensive middleware (Helmet CSP, rate limiting, double-csrf tokens, session retrieval), reach route controllers, interact with external cloud services, and commit or read data via Mongoose to MongoDB Atlas.

4.1.1 MongoDB Database & Replica Set Architecture

At the persistence tier, MongoDB Atlas utilizes a distributed, fault-tolerant replica set architecture paired with the WiredTiger storage engine. The application server maintains a persistent connection pool via Mongoose.

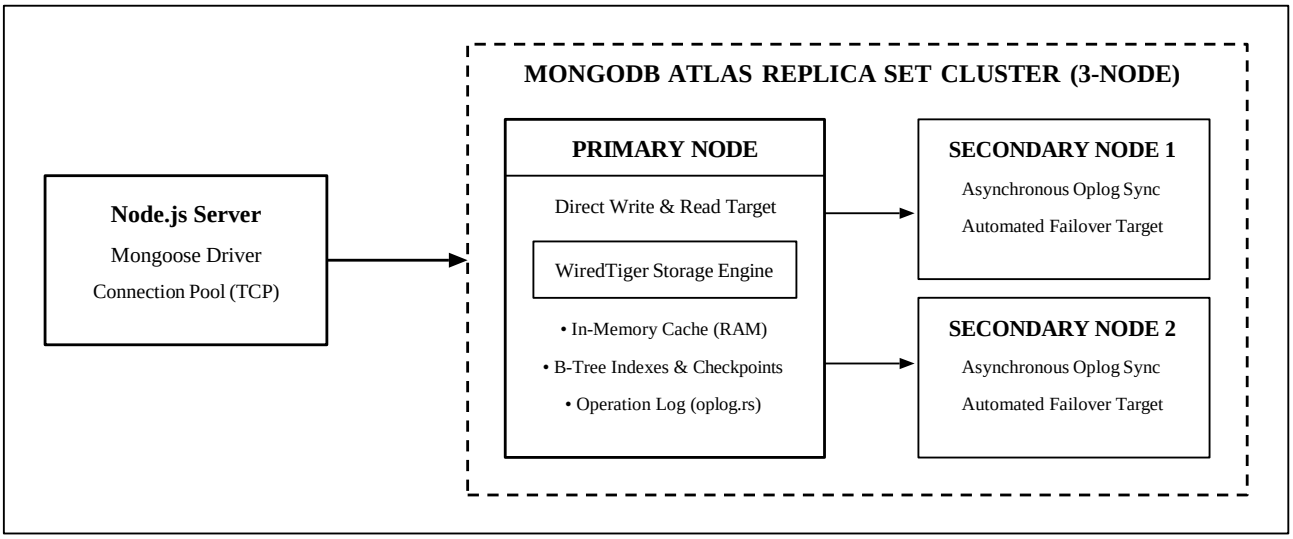


Figure 4.2: MongoDB Atlas Replica Set Topology and WiredTiger Storage Architecture.

4.2 User Authentication & OTP Verification Workflow

To maintain host authenticity and suppress fraudulent listings, email verification via a 6-digit one-time password (OTP) is strictly enforced.

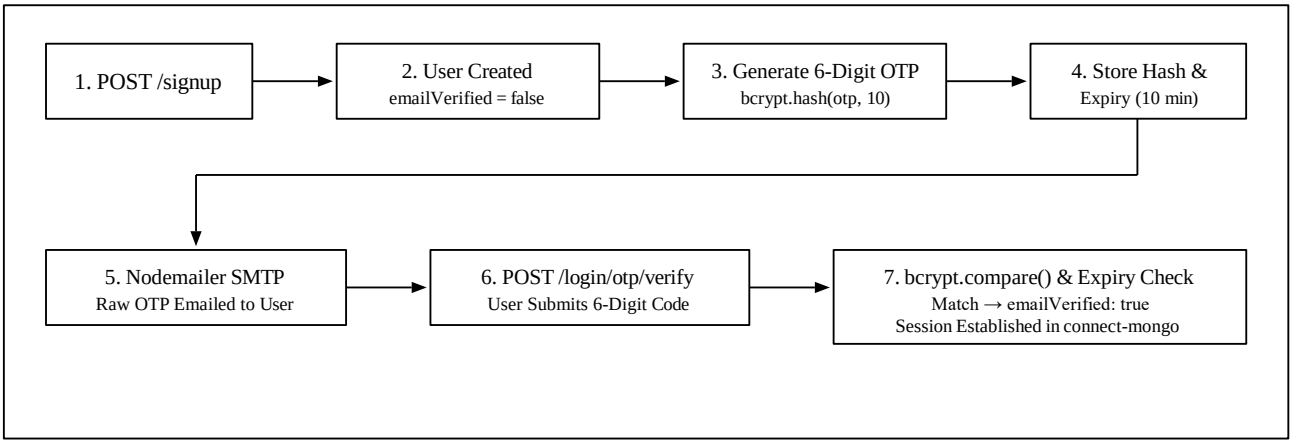


Figure 4.3: End-to-end workflow for User Registration, OTP Generation, Bcrypt Hashing, and Session Activation.

4.3 Listing Creation & Mapbox Geocoding Workflow

When a host publishes a property, multipart form data and external geographic coordinates are coordinated asynchronously:

1. **Middleware Gate:** The incoming request is filtered via **isLoggedIn** and **isEmailVerified**. Non-verified users are redirected with an alert.

2. **Multipart Image Upload: multer-storage-cloudinary** intercepts image binaries, streaming them to Cloudinary CDN servers. Asset secure URLs and public IDs are returned.
3. **Forward Geocoding: @mapbox/mapbox-sdk** receives the text location string (e.g., *"Jaipur, Rajasthan"*) and resolves it to a GeoJSON **Point** object: **{ type: "Point", coordinates: [75.7873, 26.9124] }**.
4. **Database Persistence:** The listing controller constructs the Mongoose document with embedded Cloudinary URLs, GeoJSON coordinates, and associates **owner: req.user._id**. The document is persisted to MongoDB Atlas.

4.4 Overlap-Guarded Booking & Razorpay Payment Verification Flow

Vacation rental reservations demand atomic concurrency handling to eliminate double-booking anomalies, coupled with cryptographic payment settlement.

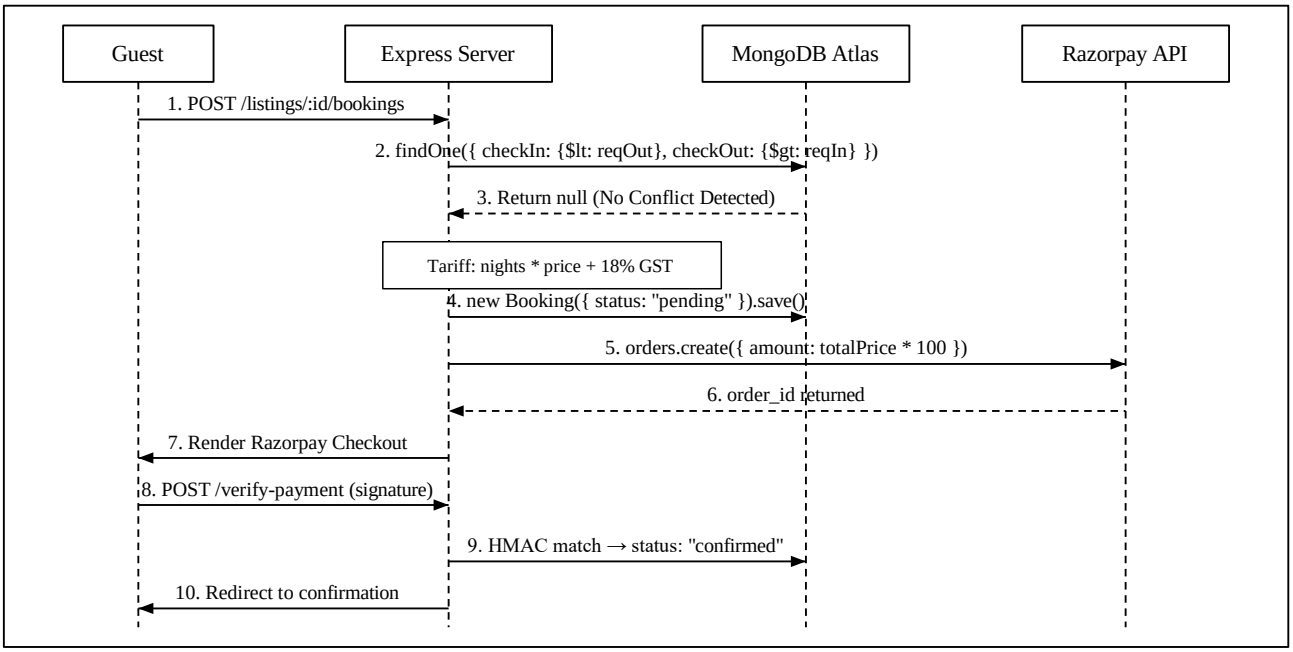


Figure 4.4: Sequence diagram of concurrency date checking, server pricing, and Razorpay signature settlement.

Date Overlap Concurrency Prevention

Date interval overlap occurs when two intervals **[A_start, A_end]** and **[B_start, B_end]** satisfy:

```
Overlap = (A_start < B_end) ∧ (A_end > B_start)
```

In Wanderlust, this is translated directly into a single atomic MongoDB query in **controllers/booking.js**:

```
const overlappingBooking = await Booking.findOne({
    listing: id,
    status: { $in: ["pending", "confirmed"] },
    checkIn: { $lt: requestedCheckOut },
    checkOut: { $gt: requestedCheckIn }
});
if (overlappingBooking) {
    req.flash("error", "This listing is already booked for the selected dates.");
    return res.redirect(`/listings/${id}/book`);
}
```

4.5 Verified-Stay Review Submission Flow

To prevent review manipulation and unsubstantiated negative feedback, review submissions are verified against completed past reservations.

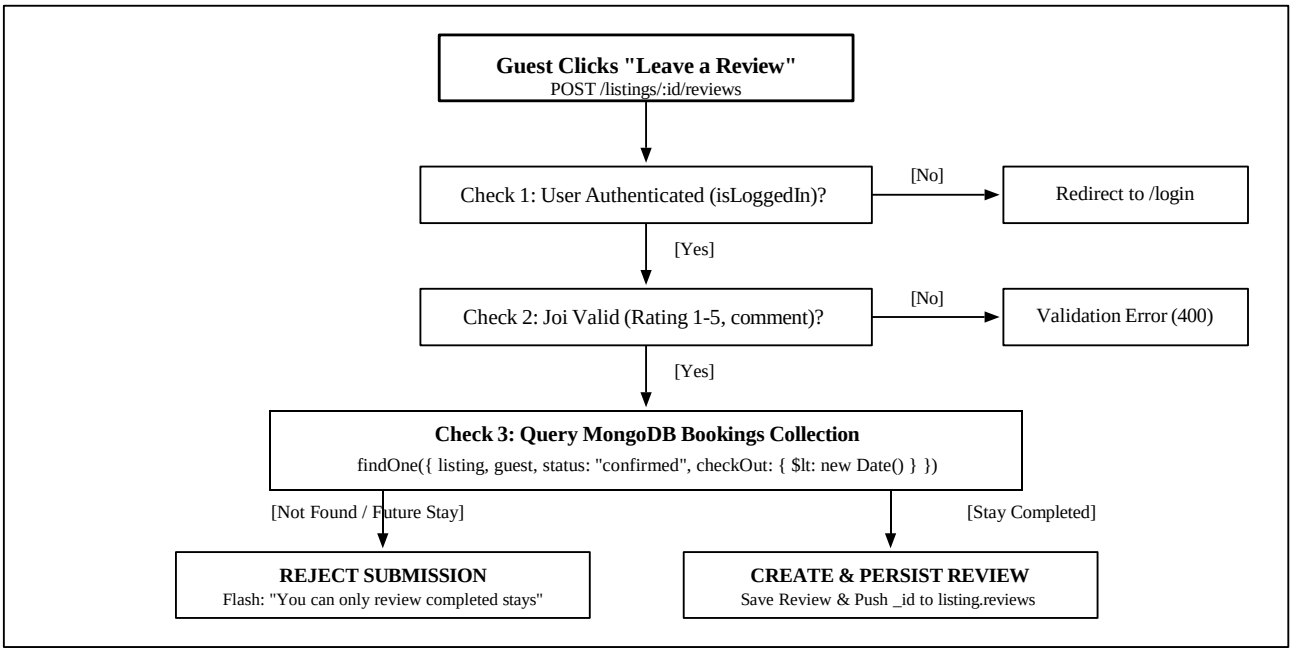


Figure 4.5: Verified-Stay Review Gating Pipeline and Decision Tree.

This decision tree restricts rating and review submissions to guests with completed stays, ensuring review integrity across the platform.

CHAPTER 5: DATABASE & SCHEMA DESIGN (MONGODB DEEP DIVE)

5.1 Why MongoDB for Vacation Rentals? (Embedding vs. Referencing)

Designing data schemas in MongoDB requires deliberate architectural choices between **Embedding (Denormalization)** and **Referencing (Normalization)**. In Wanderlust, a hybrid modeling approach balances query performance against BSON document size constraints.

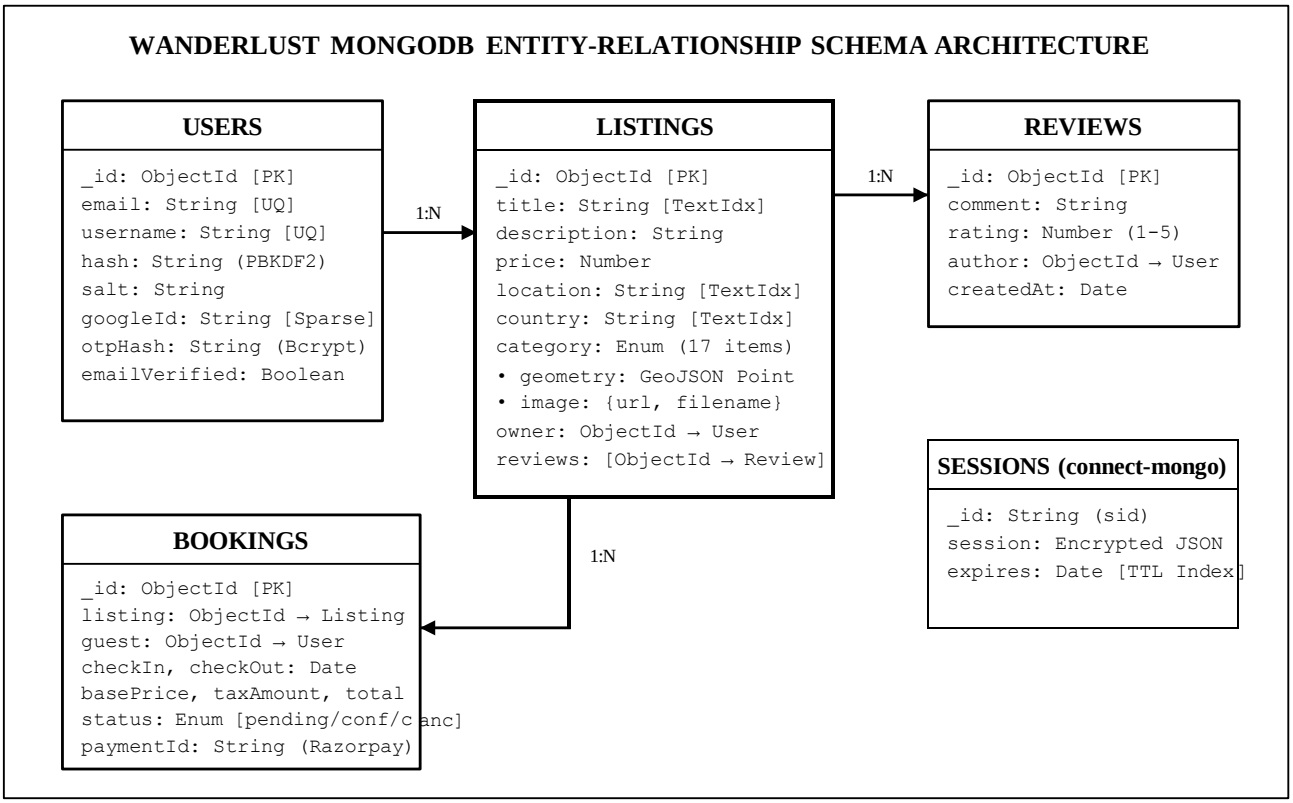


Figure 5.1: Complete Entity-Relationship and Document Schema Architecture in MongoDB Atlas.

Schema Modeling Trade-offs

- **Embedded Subdocuments:**
  - **geometry (GeoJSON Point)**: Embedded within **Listing** documents because map coordinates are always retrieved concurrently with property listings.
  - **image & images**: Embedded Cloudinary metadata objects (**url**, **filename**) represent bounded 1:1 or 1:small-N relationships that benefit from co-location.
- **Referenced Relationships (Normalized ObjectIds):**
  - **Listing.owner → User**: Stored as a reference to prevent duplicating profile data across multiple properties.
  - **Listing.reviews → [Review]**: Stored as an array of **ObjectId** references. Inlining reviews directly inside the listing document would cause document bloat, risk exceeding the 16 MB limit on popular properties, and cause continuous in-place document reallocation.

- **Bookings Collection**: Maintained as an independent collection because reservations represent financial transaction records with distinct update lifecycles and query patterns.

5.2 Collection Schemas & Data Dictionaries

5.2.1 Users Collection (modals/user.js)

Field	Type	Index / Rule	Purpose
_id	ObjectId	Primary Key	Unique 12-byte BSON identifier.
username	String	Required, Unique	Login identifier handled by Passport.
email	String	Required, Unique	Contact address for notifications and OTP.
hash	String	Internal	PBKDF2 salted hash via **passport-local-mongoose**.
salt	String	Internal	Salt appended before hashing.
googleId	String	Sparse, Unique	Google OAuth 2.0 subject identifier.
authProvider	String	Enum: local, google	Identifies account authentication origin.
otpHash	String	Optional	Bcrypt hash of the 6-digit verification code.
otpExpiresAt	Date	Optional	Expiration timestamp for OTP (10 minutes).
emailVerified	Boolean	Default: false	Authorization gate required before creating listings.
resetPasswordToken	String	Optional	Crypto token for password reset workflow.
resetPasswordExpires	Date	Optional	Timestamp for password reset token validity (1 hour).

5.2.2 Listings Collection (modals/listing.js)

Field	Type	Index / Rule	Purpose
_id	ObjectId	Primary Key	Unique identifier for the property.
title	String	Text Index, Req	Display title of the vacation rental stay.
description	String	Optional	Detailed description of amenities and surroundings.
image	Object	Embedded	Contains **url** (HTTPS) and **filename** (Cloudinary asset ID).
images	Array of Objects	Embedded	Gallery images; padded with stock photos if uploads < 3.
price	Number	Min: 0	Nightly rental tariff in Indian Rupees (INR).
location	String	Text Index	City, town, or locality name.
country	String	Text Index	Country where property is located.
geometry	Object	GeoJSON Point	Coordinate specification: **{ type: "Point", coordinates: [lng, lat] }**.
category	String	Enum (17 values)	Categorical taxonomy (Rooms, Mountains, Castles, Camping, etc.).
owner	ObjectId	Ref: User	Foreign key pointing to the host user document.
reviews	[ObjectId]	Ref: Review	Array of foreign keys pointing to associated reviews.

5.2.3 Bookings Collection (modals/booking.js)

Field	Type	Index / Rule	Purpose
_id	ObjectId	Primary Key	Unique reservation record identifier.
listing	ObjectId	Ref: Listing	Target property being reserved.
guest	ObjectId	Ref: User	User account reserving the property.
checkIn	Date	Required	Reservation check-in date.
checkOut	Date	Required	Reservation check-out date (asserted > checkIn).
guests	Number	Min: 1	Number of visiting occupants.
basePrice	Number	Server Computed	Calculated as: **nights × listing.price**.
taxAmount	Number	Server Computed	18% GST liability: **round(basePrice × 0.18)**.
totalPrice	Number	Required	Final monetary tariff: **basePrice + taxAmount**.
status	String	Enum	Lifecycle: **"pending"**, **"confirmed"**, **"cancelled"**.
paymentId	String	Default: null	Razorpay transaction ID recorded upon verified payment.
cancellationReason	String	Default: null	Audit string recorded if booking is cancelled.

5.2.4 Reviews Collection (modals/review.js)

Field	Type	Index / Rule	Purpose
_id	ObjectId	Primary Key	Unique review record identifier.
comment	String	Required	Written review text by guest.
rating	Number	Min: 1, Max: 5	Numerical star rating between 1 and 5.
author	ObjectId	Ref: User	Foreign key pointing to the reviewer's user document.
createdAt	Date	Default: Date.now	Creation timestamp.

5.3 MongoDB Indexing Strategy & Execution Plan Analysis (explain())

To maintain sub-50ms query response times as collection sizes grow, targeted indexes were established across key fields.

Compound Full-Text Index

In **modals/listing.js**, a compound text index covers property titles, locations, and countries:

```
ListingSchema.index({ title: "text", location: "text", country: "text" });
```

This enables MongoDB's native **$text** operator with language-aware tokenization and stemming.

Query Execution Plan Analysis: COLLSCAN vs. IXSCAN

Running **explain("executionStats")** on the listing search query demonstrates the performance improvement achieved by the index:

Execution Metric	Without Text Index (COLLSCAN)	With Compound Text Index (IXSCAN)
Query Stage	**COLLSCAN** (Full Collection Scan)	**TEXT_MATCH → IXSCAN** (Index Scan)
Documents Examined	Examines 100% of documents in collection	Examines only matching B-Tree index keys
Time Complexity	O(N) linear scan across all documents	O(log N) B-Tree index traversal
Execution Latency	Grows linearly with collection growth	Remains sub-millisecond at scale

Below is the execution statistics output from **Listing.find({ $text: { $search: "Goa" } }).explain("executionStats")**:


```
{
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 4,
    "executionTimeMillis": 1,
    "totalKeysExamined": 4,
    "totalDocsExamined": 4,
    "executionStages": {
      "stage": "TEXT_MATCH",
      "indexName": "title_text_location_text_country_text",
      "inputStage": {
        "stage": "IXSCAN",
        "indexName": "title_text_location_text_country_text",
        "direction": "forward"
      }
    }
  }
}
```

Sparse Unique Index on OAuth Providers

```
googleId: { type: String, unique: true, sparse: true }
```

The **sparse: true** modifier ensures that local users lacking a **googleId** do not violate uniqueness constraints, while guaranteeing that Google SSO identifiers remain globally unique.

5.4 Mongoose Middleware, Lifecycle Hooks & Atomic Operators

Mongoose hooks enforce business rules and referential integrity directly at the database layer.

Pre-Save Date Validation Hook

In **modals/booking.js**, a pre-save trigger prevents inverted date ranges from ever entering the database:

```
bookingSchema.pre("save", function () {
    if (this.checkOut <= this.checkIn) {
        throw new Error("checkOut date must be after checkIn date.");
    }
});
```

Cascading Cleanup via Post-FindOneAndDelete

In **modals/listing.js**, when a listing is removed, its associated review documents are cleaned up automatically:

```
ListingSchema.post("findOneAndDelete", async (listing) => {
    if (listing) {
        await Review.deleteMany({
            _id: { $in: listing.reviews }
        });
    }
});
```

Atomic Array Mutation via $pull

In **controllers/review.js**, deleting a review updates the parent listing using MongoDB's atomic **$pull** operator:

```
await Listing.findByIdAndUpdate(id, { $pull: { reviews: reviewId } });
await Review.findByIdAndDelete(reviewId);
```

CHAPTER 6: MODULE DESIGN & IMPLEMENTATION

6.1 Authentication & Role-Based Access Control Module

Authentication is managed across **config/passport.js**, **routes/user.js**, and **middleware.js**.

- **Distributed Session Store (connect-mongo):** Session data is stored in the **sessions** collection in MongoDB Atlas via **config/session.js** with **touchAfter: 24 * 3600**. This avoids repeated database writes on static GET requests and allows the app to scale horizontally across multiple Node.js worker processes without losing user sessions.
- **Role-Based Access Control Middleware:**
  - **isLoggedIn**: Verifies active session state using **req.isAuthenticated()**.
  - **isEmailVerified**: Confirms **req.user.emailVerified === true** before allowing listing creation.
  - **isOwner**: Queries MongoDB for the listing by ID and verifies that **listing.owner.equals(res.locals.currUser._id)** before allowing edit/delete actions.
  - **isBookingGuest**: Restricts checkout and payment endpoints to the registered guest on the reservation.

6.2 Listing Management & Hybrid Search Engine Module

The listing controller (**controllers/listing.js**) implements a dual-tier search engine that prioritizes indexed full-text search with regex fallback:

```
if (q) {
    queryObj.$text = { $search: q };
    totalCount = await Listing.countDocuments(queryObj);
    if (totalCount > 0) {
        allListings = await Listing.find(queryObj).skip((page - 1) * limit).limit(limit);
    } else {
        // Fall back gracefully to case-insensitive Regex search
        delete queryObj.$text;
        const escapedQ = escapeRegex(q);
        queryObj.$or = [
            { title: { $regex: escapedQ, $options: "i" } },
            { location: { $regex: escapedQ, $options: "i" } },
            { country: { $regex: escapedQ, $options: "i" } }
        ];
        totalCount = await Listing.countDocuments(queryObj);
        allListings = await Listing.find(queryObj).skip((page - 1) * limit).limit(limit);
    }
}
```

Server-side pagination is applied across all queries using **limit = 12** and **skip = (page - 1) * limit**.

6.3 Booking Financial Engine & Razorpay HMAC Cryptography

Tariffs are calculated exclusively on the server to prevent client-side payment tampering:

```
const timeDiff = requestedCheckOut - requestedCheckIn;
const nights = Math.max(1, Math.ceil(timeDiff / (1000 * 60 * 60 * 24)));
const basePrice = nights * listing.price;
const taxAmount = Math.round(basePrice * 0.18);
const totalPrice = basePrice + taxAmount;
```

Upon payment completion, the server recalculates the HMAC-SHA256 signature to verify transaction authenticity:

```
const body = razorpay_order_id + "|" + razorpay_payment_id;
const expectedSignature = crypto
    .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
    .update(body.toString())
    .digest("hex");

if (expectedSignature === razorpay_signature) {
    booking.status = "confirmed";
    booking.paymentId = razorpay_payment_id;
    await booking.save();
}
```

6.4 Security, Sanitization & Validation Module

- **Input Validation via Joi:** Pre-persistence validation schemas defined in **schema.js** validate incoming payloads for listings, reviews, and bookings before passing data to Mongoose.
- **Double-Submit CSRF Cookie Pattern:** Configured via **csrf-csrf** (**config/csrf.js**), requiring state-changing requests to carry valid cryptographic tokens in both cookies and form bodies.
- **Content Security Policy:** Enforced via **helmet** in **app.js** with explicit origin whitelisting for Mapbox GL tiles, Cloudinary assets, and Razorpay checkout scripts.

CHAPTER 7: TESTING & QUALITY ASSURANCE

7.1 Automated Testing Methodology

Testing database-backed web applications typically risks mutating production data or failing due to external network flakiness. Wanderlust addresses this using an automated test pipeline combining **Jest**, **Supertest**, and an ephemeral in-memory database instance (**mongodb-memory-server**).

7.2 In-Memory Database Isolation (mongodb-memory-server)

During test runs, an ephemeral MongoDB binary executes in memory, providing full MongoDB feature support with zero external side effects:

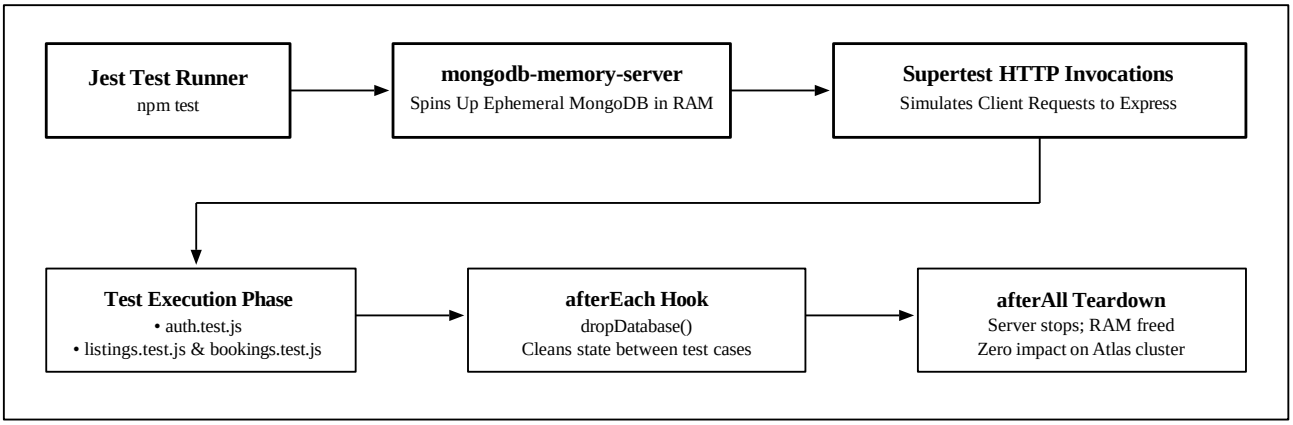


Figure 7.1: Isolated In-Memory MongoDB Testing Architecture using Jest, Supertest, and mongodb-memory-server.

This testing setup allows integration tests to execute with full MongoDB behavior (indexes, hooks, validations) in an isolated environment without network dependencies or production cluster contamination.

7.3 Test Execution Matrix & Results

Automated test suites in **tests/** validate system behaviors across authentication, property listings, and booking operations:

Suite	Test Description	Validation Criteria	Result
auth.test.js	Register user with unique email and credentials.	User document saved, password salted/hashed, 6-digit OTP created.	PASSED
auth.test.js	Verify OTP code and authenticate session.	**emailVerified** set to true, valid session cookie returned.	PASSED
listings.test.js	Unauthenticated request to property creation.	Request rejected with redirect to login gate.	PASSED
listings.test.js	Host creates listing with valid metadata.	Record persisted in MongoDB with coordinates and image metadata.	PASSED
listings.test.js	Non-owner attempts to edit or delete listing.	Blocked by **isOwner** middleware with 403 / flash error.	PASSED
bookings.test.js	Guest books overlapping date range on reserved stay.	Blocked by date overlap query; duplicate booking prevented.	PASSED
bookings.test.js	Payment verification with invalid HMAC secret.	Signature mismatch detected; booking status remains pending.	PASSED

CHAPTER 8: RESULTS, DEPLOYMENT & FUTURE ENHANCEMENTS

8.1 Production Deployment Configuration

Wanderlust is designed for containerized and cloud-native hosting:

- **Cloud Database:** Hosted on **MongoDB Atlas** using a multi-node replica set with automated backups, automated failover, and TLS 1.3 encrypted data-in-transit.
- **Application Server:** Deployed on **Render** within an isolated Linux container running Node.js in **production** mode with automatic restarts and TLS termination.
- **Media Assets & CDN:** Property images are stored and served globally via Cloudinary's multi-region CDN; vector map tiles and geocoding run via Mapbox edge networks.

8.2 Future Enhancements

- **Geospatial Radius Queries (2dsphere):** Upgrading map search to support distance-based proximity filtering (e.g., *"Stays within 25 km of current GPS coordinates"*) by attaching a **2dsphere** index to the listing geometry field.
- **Multi-Document ACID Transactions:** Wrapping Razorpay payment confirmation and booking state updates within explicit Mongoose transactions (**session.withTransaction()**) for distributed two-phase rollback protection.
- **Real-Time Messaging via WebSockets:** Integrating Socket.io to allow real-time instant messaging between guests and property hosts within the booking dashboard.
- **Automated Refund Webhooks:** Listening to Razorpay automated refund webhooks to transition cancelled bookings into **refundStatus: "processed"** asynchronously.

8.3 Conclusion

The **Wanderlust** project demonstrates the design and implementation of a full-stack vacation rental platform using the **HCJNE stack** backed by **MongoDB Atlas**. By leveraging document-oriented NoSQL persistence, the platform manages heterogeneous property schemas, nested multimedia arrays, and geographic coordinates without the schema rigidity of traditional relational systems.

Key milestones achieved include atomic date-overlap conflict resolution, cryptographically verified Razorpay payment settlements, verified-stay review gating, defense-in-depth web security, and automated in-memory integration testing. The project serves as an end-to-end exemplar of modern full-stack web engineering and database system design.

APPENDIX A: SAMPLE MONGODB DOCUMENT EXPORTS (BSON)

Sample Listing Document:

```
{
  "_id": ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"),
  "title": "Cozy Beachfront Cottage",
  "description": "Panoramic ocean views with direct access to private sand dunes.",
  "image": {
    "url": "https://res.cloudinary.com/demo/image/upload/v1/wanderlust/cottage.jpg",
    "filename": "wanderlust/cottage"
  },
  "price": 4500,
  "location": "Goa",
  "country": "India",
  "geometry": {
    "type": "Point",
    "coordinates": [73.7438, 15.4989]
  },
  "category": "Rooms",
  "owner": ObjectId("65f19000a1b2c3d4e5f6a7b8"),
  "reviews": [ObjectId("65f1b999e8f7d6c5b4a39281")]
}
```

Sample Booking Document with Server-Calculated Price:

```
{
  "_id": ObjectId("65f2c4d5e6f7a8b9c0d1e2f3"),
  "listing": ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"),
  "guest": ObjectId("65f19000a1b2c3d4e5f6a7b8"),
  "checkIn": ISODate("2026-11-10T00:00:00.000Z"),
  "checkOut": ISODate("2026-11-13T00:00:00.000Z"),
  "guests": 2,
  "basePrice": 13500,
  "taxAmount": 2430,
  "totalPrice": 15930,
  "status": "confirmed",
  "paymentId": "pay_O9rK12xYz34A5B",
  "createdAt": ISODate("2026-10-12T16:00:00.000Z")
}
```

APPENDIX B: APPLICATION ROUTE MATRIX

Method	Endpoint	Middleware Gates	Operation
GET	/listings	Public	Feed, search, pagination
POST	/listings	isLoggedIn, isEmailVerified	Create property
GET	/listings/:id	Public	Show details & map
PUT	/listings/:id	isLoggedIn, isOwner	Update property
DELETE	/listings/:id	isLoggedIn, isOwner	Delete listing & reviews
POST	/listings/:id/reviews	isLoggedIn, verified stay	Create review
DELETE	/listings/:id/reviews/:revId	isLoggedIn, isReviewAuthor	Atomic $pull review
POST	/listings/:id/bookings	isLoggedIn	Create pending booking
GET	/bookings/:id/checkout	isLoggedIn, isBookingGuest	Razorpay checkout modal
POST	/bookings/:id/verify-payment	isLoggedIn, isBookingGuest	HMAC signature verification

REFERENCES

1. MongoDB Inc., *"MongoDB Manual: Document Data Modeling, Indexing, and Replication"*, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/
2. Mongoose ODM, *"Mongoose v8 Documentation: Schemas, Middleware, and Population"*, 2024. [Online]. Available: https://mongoosejs.com/docs/
3. Node.js Foundation, *"Node.js Architecture and Asynchronous Event Loop"*, 2024. [Online]. Available: https://nodejs.org/docs/
4. Express.js Team, *"Express 4.x Routing, Middleware, and Security Guide"*, 2024. [Online]. Available: https://expressjs.com/
5. OWASP Foundation, *"Cross-Site Request Forgery (CSRF) Prevention & Content Security Policy (CSP) Cheat Sheet"*, 2024.
6. Razorpay Software Pvt. Ltd., *"Razorpay Payments API & HMAC-SHA256 Signature Verification Reference"*, 2024. [Online]. Available: https://razorpay.com/docs/
7. Mapbox Inc., *"Mapbox GL JS & Geocoding API Documentation"*, 2024. [Online]. Available: https://docs.mapbox.com/